

UNIT 5 · WEEK 7

# Pipes and process composition

📅 Week 7 · dates to be confirmed    </> Lab TD4 (solved + AI challenge)    ~ 2 animations

LAB TD4 · SOLVED

## Unnamed pipes and process composition

How the shell builds `ls | grep | wc`: one pipe per `|`, one process per command, `dup2` to plug the output of one into the input of the next and, above all, closing in time the ends that are not used.

✓ **Assignment: lab report**

Hand in this lab with a short LaTeX report (just what is needed). What it must cover, the template, an example and the Prism guide are in the Assignment tab. Generative AI allowed, with a declaration.

### EXERCISES

- |   |                                                    |   |                                    |
|---|----------------------------------------------------|---|------------------------------------|
| 1 | A pipe between parent and child                    | 3 | <code>ps</code> in uppercase       |
| 2 | The rules of pipes: capacity, blocking and SIGPIPE | 4 | A fixed command pipeline           |
|   |                                                    | 5 | The pipeline from the command line |

TD4 · 1

● Solved · tested

### A pipe between parent and child

**Statement.** Create a pipe with `pipe(2)` and duplicate the process. The parent writes "Hello World" 10 times, says on standard error that it is leaving and ends. The child reads at most 500 bytes in a single read, prints them, says so and ends. Use macros for the constants.

#### Solution

```
C tds/td4/mypipe.c
```

```
/*
 * TD4 - exercises 1 and 2: an unnamed pipe between a parent (writer) and a
 * child (reader).
```

```

* ./mypipe [count]    the parent writes "Hello World\n" `count` times
(default 10)
* The child reads AT MOST 500 bytes, once, prints them and leaves.
*
* Exercise 2: with a large count (10000 x 12 bytes = 120 KB > 64 KiB, the
pipe capacity)
* the parent blocks on write() once the pipe is full. When the child exits,
nobody can read
* any more: the next write() raises SIGPIPE and kills the parent ("Broken
pipe", status 141
* in the shell). That only works if the parent CLOSED its own read end: with
* -DKEEP_READ_END (Makefile target mypipe_bug) the parent itself is still a
potential reader,
* the kernel never raises SIGPIPE and the parent blocks forever.
*/
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/wait.h>
#include <unistd.h>

#include "utils.h"

#define MESSAGE    "Hello World\n"
#define READ_MAX   500

int main(int argc, char *argv[])
{
    int count = argc > 1 ? atoi(argv[1]) : 10;
    int fds[2];                                /* fds[0] read end,
fds[1] write end */

    exit_if(pipe(fds) == -1, "pipe");
    pid_t pid = fork();
    exit_if(pid == -1, "fork");

    if (pid == 0) {                            /* child: reader */
        exit_if(close(fds[1]) == -1, "close"); /* never writes */
        char buffer[READ_MAX];
        ssize_t n = read(fds[0], buffer, READ_MAX); /* one single read */
        exit_if(n == -1, "read");
        exit_if(write_all(STDOUT_FILENO, buffer, (size_t)n) == -1, "write");
        fprintf(stderr, "I am the child, I read %zd bytes and I leave\n", n);
        exit_if(close(fds[0]) == -1, "close");
        return EXIT_SUCCESS;
    }

```

```

}

/* parent: writer */
#ifdef KEEP_READ_END
    exit_if(close(fds[0]) == -1, "close");          /* THE line that
exercise 2 is about */
#endif
for (int i = 0; i < count; i++)
    exit_if(write_all(fds[1], MESSAGE, strlen(MESSAGE)) == -1, "write");
fprintf(stderr, "I am the parent, I wrote %d messages and I leave\n",
count);
exit_if(close(fds[1]) == -1, "close");
exit_if(waitpid(pid, NULL, 0) == -1, "waitpid");
return EXIT_SUCCESS;
}

```

### Real output

```

$ ./mypipe
I am the parent, I wrote 10 messages and I leave
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
I am the child, I read 120 bytes and I leave

```

`fds[0]` is the read end and `fds[1]` the write end (like 0 = input and 1 = output). After `fork` both processes hold both ends: each one closes the end it does not use.

TD4 · 2

● Solved · real outputs

## The rules of pipes: capacity, blocking and SIGPIPE

**Statement.** Pass as an argument how many times the parent writes: 50, 100, 500, 1000, 5000, 10000... If the program seems to hang, check the state of the processes, draw the

descriptor tables and fix it. With the fixed program, what is its exit status?

## Solution

### Real output

```
$ python3 -c 'import os, fcntl; r, w = os.pipe(); print(fcntl.fcntl(w,
fcntl.F_GETPIPE_SZ), "bytes")'
65536 bytes
```

### Real output

```
$ timeout 3 ./mypipe_bug 10000 > /dev/null; echo "exit status: $? (124 =
killed by timeout: it was blocked)"
I am the child, I read 500 bytes and I leave
exit status: 124 (124 = killed by timeout: it was blocked)
```

### Real output

```
$ ./mypipe 10000 > /dev/null; echo "exit status of mypipe: $?"
I am the child, I read 500 bytes and I leave
exit status of mypipe: 141
```

**New: the race, measured.** The original statement suggested finding "from how many messages on" the parent dies. On a multi-core machine there is no fixed threshold: the child reads 500 bytes and leaves while the parent is still writing, and who wins depends on the scheduler. 30 runs per size:

### Real output

```
$ for n in 50 100 ... 1000; do run ./mypipe n 30 times; count exit statuses;
done
  50 messages: 30 x status 0      0 x status 141 (SIGPIPE)
 100 messages: 30 x status 0      0 x status 141 (SIGPIPE)
 200 messages: 28 x status 0      2 x status 141 (SIGPIPE)
 300 messages: 19 x status 0     11 x status 141 (SIGPIPE)
 400 messages:  5 x status 0     25 x status 141 (SIGPIPE)
 500 messages:  5 x status 0     25 x status 141 (SIGPIPE)
1000 messages:  0 x status 0     30 x status 141 (SIGPIPE)
```

**See it animated** → [A5.2](#)

1. The pipe has a limited capacity: **65,536 bytes** by default on Linux.  $10,000 \times 12$  bytes = 120,000 bytes do not fit: when it fills up, `write` blocks the parent until someone reads.
2. The child reads 500 bytes and ends. If the parent **did not close** its own read end (`mypipe_bug`), the kernel sees that a reader still exists —the parent itself— and leaves it blocked forever: `timeout` kills it after 3 s (status 124).
3. With the end closed, once the child leaves there are no readers: the next `write` gets the **SIGPIPE** signal, whose default action is to terminate the process. The shell shows "Broken pipe" and status  $141 = 128 + 13$ .

Animation A5.2 · Pipe capacity →

TD4 · 3

● Solved · tested

## ps in uppercase

**Statement.** The child keeps reading from the pipe and prints what it reads in uppercase. The parent redirects its standard output to the pipe with `dup2` and runs `ps` with `execlp`.

### Solution

```
C tds/td4/upper.c

/*
 * TD4 - exercise 3: ps | uppercase, built by hand.
 * The child reads the pipe until end of file and prints it in uppercase.
 * The parent redirects its stdout to the pipe (dup2) and becomes `ps`
 * (execlp).
 * The child sees end of file when `ps` exits, because it was the last
 * writer.
 */
#include <ctype.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

#define BUFFER_SIZE 512

static void to_upper(int fd_in, int fd_out)
```

```

{
    char buffer[BUFFER_SIZE];
    ssize_t n;

    while ((n = read(fd_in, buffer, sizeof buffer)) > 0) {      /* until end
of file */
        for (ssize_t i = 0; i < n; i++)
            buffer[i] = (char)toupper((unsigned char)buffer[i]);
        exit_if(write_all(fd_out, buffer, (size_t)n) == -1, "write");
    }
    exit_if(n == -1, "read");
}

int main(void)
{
    int fds[2];

    exit_if(pipe(fds) == -1, "pipe");
    pid_t pid = fork();
    exit_if(pid == -1, "fork");

    if (pid == 0) {                                             /* child */
        exit_if(close(fds[1]) == -1, "close");                /* otherwise it never sees
end of file */
        to_upper(fds[0], STDOUT_FILENO);
        return EXIT_SUCCESS;
    }

    /* parent */
    exit_if(close(fds[0]) == -1, "close");
    exit_if(dup2(fds[1], STDOUT_FILENO) == -1, "dup2");
    exit_if(close(fds[1]) == -1, "close");                    /* 1 already refers to the
pipe */
    execlp("ps", "ps", (char *)NULL);                        /* "ps" twice: file to run,
then argv[0] */
    exit_if(1, "execlp ps");
    return EXIT_FAILURE;
}

```

### Real output

```
$ ./upper | head -4
  PID TTY          TIME CMD
 6706 ?           00:00:25 UVICORN
 9771 ?           00:00:03 SYSTEMD
 9801 ?           00:00:00 (SD-PAM)
```

The child ends when `read` returns 0, and that only happens when *all* write ends are closed: the one of `ps` (when it ends) and its own, which is why it closes it at the start. `ps` does not know it writes into a pipe: its descriptor 1 simply points to it.

TD4 · 4

● Solved · tested

## A fixed command pipeline

**Statement.** Run the pipeline defined in a `commands` variable (for instance `ls -l | tr -d '[:blank:]'`), with any number of commands. The program ends when the last one ends and returns its status; if it died from a signal, it reports it and returns `EXIT_FAILURE`.

### Solution

C tds/td4/pipeline.c

```
/*
 * TD4 - exercise 4: run a fixed pipeline, like the shell does for ls -l |
 * tr -d '[:blank:]'
 * Works for any number of commands. Returns the exit status of the LAST
 * command, or
 * EXIT_FAILURE (with a message) if it was killed by a signal.
 */
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>

#include "utils.h"

static char *ls[] = { "ls", "-l", NULL };
static char *tr[] = { "tr", "-d", "[:blank:]", NULL };
```

```

static char **commands[] = { ls, tr };
static const int ncommands = sizeof commands / sizeof commands[0];

int main(void)
{
    int in = STDIN_FILENO;      /* where the next command reads from */
    pid_t last = -1;

    for (int i = 0; i < ncommands; i++) {
        int fds[2] = { -1, -1 };
        if (i < ncommands - 1) /* every command but
the last writes to a pipe */
            exit_if(pipe(fds) == -1, "pipe");

        pid_t pid = fork();
        exit_if(pid == -1, "fork");
        if (pid == 0) {
            if (in != STDIN_FILENO) { /* not the first: read
from previous pipe */
                exit_if(dup2(in, STDIN_FILENO) == -1, "dup2 stdin");
                exit_if(close(in) == -1, "close");
            }
            if (fds[1] != -1) { /* not the last: write
to the new pipe */
                exit_if(dup2(fds[1], STDOUT_FILENO) == -1, "dup2 stdout");
                exit_if(close(fds[0]) == -1 || close(fds[1]) == -1, "close");
            }
            execvp(commands[i][0], commands[i]);
            perror(commands[i][0]);
            _exit(127);
        }

        /* parent: close what the child inherited, keep only the read end for
the next one */
        if (in != STDIN_FILENO)
            exit_if(close(in) == -1, "close");
        if (fds[1] != -1) {
            exit_if(close(fds[1]) == -1, "close");
            in = fds[0];
        }
        last = pid;
    }

    /* Wait for every child (no zombies), remember the status of the last
one. */

```

```
int status, last_status = 0;
pid_t pid;
while ((pid = wait(&status)) > 0)
    if (pid == last)
        last_status = status;

if (WIFEXITED(last_status))
    return WEXITSTATUS(last_status);
if (WIFSIGNALED(last_status))
    fprintf(stderr, "pipeline: last command killed by %s\n",
    signame(WTERMSIG(last_status)));
return EXIT_FAILURE;
}
```

#### Real output

```
$ cd demo && ../pipeline
total8
-rw-rw-r--1studentstudent4Sep3013:13one.txt
-rw-rw-r--1studentstudent2Sep3013:13two.txt
```

The pattern: each round creates the *next* pipe (except for the last command); the child takes the previous pipe ( `in` ) as input and the new one as output; the parent closes what the child already inherited and keeps the read end for the next round. At the end the parent waits for **all** children (otherwise they would stay zombies) and keeps the status of the last one.

TD4 · 5

● Solved · tested

## The pipeline from the command line

**Statement.** Like the previous one, but the commands come from the command line separated by `--`: `pipeline ls -l -- grep a -- wc -l` is equivalent to `ls -l | grep a | wc -l`.

### Solution



C tds/td4/pipeline\_args.c

```

int main(int argc, char *argv[])
{
    if (argc < 2) {
        fprintf(stderr, "usage: %s cmd [args] [-- cmd [args]]...\n",
argv[0]);
        return EXIT_FAILURE;
    }

    /* Split argv in place. commands[k] points to the first word of command
k. */
    char **commands[argc];
    int ncommands = 0;
    commands[ncommands++] = &argv[1];
    for (int i = 1; i < argc; i++) {
        if (strcmp(argv[i], "--") == 0) {
            argv[i] = NULL;                /* ends the previous
command */
            if (i + 1 ≥ argc || strcmp(argv[i + 1], "--") == 0 ||
commands[ncommands - 1][0] == NULL) {
                fprintf(stderr, "%s: empty command\n", argv[0]);
                return EXIT_FAILURE;
            }
            commands[ncommands++] = &argv[i + 1];
        }
    }
}

```

#### Real output

```

$ ./pipeline_args ls -l demo -- grep txt -- wc -l
2

```

#### Real output

```

$ ./pipeline_args ls -- sh -c 'exit 3'; echo "status: $?"
status: 3

```

#### Real output

```

$ ./pipeline_args ls -- sh -c 'kill -TERM $$'
pipeline: last command killed by SIGTERM
[exit status 1]

```

Trick: `argv` is already a `NULL`-terminated array of strings. Replacing every `--` with `NULL`, each command becomes a slice of `argv` ready for `execvp`, without copying anything. The rest is identical to exercise 4.

## Checklist

### Generative AI challenge · Pipes in your mini shell



**Goal.** Add the `|` operator to the mini shell by reusing `pipeline_args.c`, and diagnose with the AI a deadlock caused on purpose.

**Starting prompt.** Use it in the tool you prefer (Claude, ChatGPT, Gemini, Copilot, Codex...) with the *solved* version of the lab as context. Ask for a plan first and review every step before accepting it.

#### Prompt

I have a mini shell and a program that runs pipelines of N commands [paste both].  
(1) Help me integrate the `|` operator into the shell. (2) Then I will remove one `close()` in the parent on purpose: explain, drawing the descriptor tables of every process, why ``ls | wc -l`` no longer ends, and how you would confirm it with ``ls -l /proc/<pid>/fd``. Do not fix the code yourself: guide me with questions.

#### How to check the result

- ``ls -l | grep c | wc -l`` gives the same result as bash.
- With the `close()` removed the command hangs and ``/proc/<pid>/fd`` shows the write end still open.
- You can name which process held the descriptor that prevented end of file.

**Declare your use of AI:** tool, relevant prompts, what you checked yourself and what you fixed. What counts is that you can explain and verify every decision.